

Outdoor Vision CV App

Intake Project Tool - Product Requirements Document

VERSION	DATE	STATUS
1.0	September 1, 2026	Pending Approval
OWNER	APPROVER	PLATFORM
Dave Vercher	Dave Vercher	Windows Desktop

DOCUMENT
INTENT

Implementation-ready requirements for integrating the legacy JPEG sorting workflow as the first reusable project tool in the Outdoor Vision CV App. Intake will copy recursively discovered JPEGs into a project-managed queue, support class-based keyboard sorting, and establish intake/sorted as the project's master dataset.

This document is the authoritative requirements baseline for the Intake project-tool release.

Contents

1. Document Purpose	3
2. Product Vision and Scope	3
2.1 In Scope	3
2.2 Out of Scope	3
3. Terminology and Filesystem Model	3
3.1 Terminology	3
3.2 Required Project Structure	4
4. Project Tool Entry	4
5. Intake Landing Screen	4
6. Source Selection and Discovery	4
7. Import Preview and Confirmation	5
8. Copy Behavior and Filename Collisions	5
9. Import Failures, Cancellation, and Results	5
10. Minimal Intake Manifest	6
10.1 Manifest Recovery	6
11. Queue Ordering and Resume	6
12. Class Discovery and Validation	6
13. Class Management and Key Mapping	6
14. Class Deletion	7
15. Sorting Screen and Image Viewer	7
16. Classification, Skip, and Focus Safety	7
17. Multi-Step Undo and Session Boundaries	8
18. Completion and Review Master Dataset	8
18.1 Completion	8
18.2 Review Master Dataset	8
19. Project Statistics and Last Edited	8
20. Loading, Error, and Accessibility Requirements	8
21. State Transitions	9
22. Suggested Architecture Boundaries	9
23. Acceptance Criteria	9
24. Required Tests	10
24.1 Automated	10
24.2 Manual GUI Verification	11
25. Risks, Assumptions, and Future Enhancements	11

1. Document Purpose

This document defines the product, user-experience, filesystem, persistence, safety, and quality requirements for Intake, the first project-level tool in the Outdoor Vision CV App. It is intended to be sufficient implementation context for a future coding session and must be read alongside the current repository.

The implementation must integrate the successful behavior of the existing standalone `jpg_sorter.py` into the project experience while preserving the existing home page, project registry, library relocation, statistics, and legacy sorter launch path.

2. Product Vision and Scope

Outdoor Vision CV is a reusable local computer-vision data-preparation pipeline for Windows. Intake is the first pipeline stage: it brings JPEGs into a project, establishes class folders, and lets a user rapidly sort images into a master dataset without altering the original source folder.

2.1 In Scope

- A clickable Intake tool card on the project screen.
- Recursive discovery and validation of JPEG files from one dropped or browsed source folder.
- Copying valid images into a flattened, project-managed unsorted queue.
- Import preview, progress, cancellation, partial-failure reporting, and results summaries.
- Class-folder creation and management beneath intake/sorted.
- Manual 0-9 key assignment and responsive keyboard/button sorting.
- Multi-step session Undo, Skip, external viewing, and resumable queues.
- A summary-only Review Master Dataset screen.
- A minimal private Intake manifest for import batches and queue order.
- Project statistics integration, tests, and README documentation.

2.2 Out of Scope

- Transactional staging folders and automatic rollback of partial imports.
- Broad cross-app image identity, lineage, annotations, or metadata tracking.
- Duplicate-content detection or content fingerprints.
- PNG, BMP, TIFF, HEIC, RAW, video, or other non-JPEG imports.
- Advanced dataset browsing, thumbnails, reclassification, or editing from Review.
- Nested class taxonomies, annotation, model training, export, or cloud services.
- Deprecation or removal of `jpg_sorter.py` or `run_jpg_sorter.bat`.
- Guaranteed network, UNC, removable, or mapped-drive support; only local Windows folders are supported.

RELEASE BOUNDARY

The existing standalone JPEG sorter and launcher remain functional and unchanged. Intake is an integrated project tool, not a replacement-removal task for this release.

3. Terminology and Filesystem Model

3.1 Terminology

- Intake: the project tool specified by this document.
- Source folder: the single external folder selected or dropped for one import.

- Import batch: the valid JPEG files accepted from one confirmed source-folder import.
- Unsorted queue: manifest-registered images in intake/unsorted awaiting classification.
- Class: a direct child folder of intake/sorted.
- Mapped class: a class assigned to one unique number key from 0 through 9.
- Master dataset: the complete physical contents of intake/sorted.
- Intake session: the period from Start/Resume Sorting until the user leaves Intake or closes the app.

3.2 Required Project Structure

Intake must create the following structure when the tool is first opened. Existing folders must be reused without overwriting their contents.

```
<Project Folder>/
  intake/
    unsorted/
    sorted/
    <Class Name>/
```

- Imported JPEGs are flattened into intake/unsorted; source subfolders are not reproduced.
- Class folders contain images directly. Nested folders are ignored by class counts and produce a non-blocking warning.
- All project-visible user files stay beneath the project folder. Private Intake records do not.

4. Project Tool Entry

Opening a normal project must show a project-tools view. Intake appears as a clickable card with the title Intake and this short description: Import JPEG images into the project, create classes, and sort images into the master dataset.

- The entire card is the open action; it has no status, metrics, or separate Open button.
- The project page must retain a clear route back to the project list.
- Intake must use the current compact, Obsidian-inspired dark theme, established typography, spacing, controls, and focus treatment.

5. Intake Landing Screen

The landing screen is the control center for import, classes, mappings, review, and queue entry. It must contain:

- A genuine Windows folder drop zone and a Browse for Folder action.
- The current direct class folders and ten mapping slots in numeric 0-9 order.
- A separate Unmapped Classes list with Assign Key actions.
- New Class, Rename, Delete, and Assign/Change Key actions.
- The number of manifest-registered unsorted images currently present on disk.
- Start/Resume Sorting, Review Master Dataset, and navigation back to the project.

Start/Resume Sorting is disabled when the queue is empty or when no class has a valid key assignment. Guidance must explain the unmet condition.

6. Source Selection and Discovery

ID	Requirement	Priority
IMP-01	Accept exactly one source folder per import through either real folder drag-and-drop or the standard Windows folder picker.	Must
IMP-02	Reject multiple folders, individual files, or mixed file/folder drops with: Select or drop one folder at a time.	Must
IMP-03	Recursively discover case-insensitive .jpg and .jpeg candidates in all nested folders.	Must
IMP-04	Ignore hidden/system items and directory symlinks, junctions, and reparse points. Show only an aggregate skipped count for hidden/system items.	Must
IMP-05	Validate candidate contents as JPEG images by opening/verifying them; do not trust extensions alone.	Must
IMP-06	Do not copy, modify, rename, or delete source files during discovery or preview.	Must
IMP-07	Run discovery without freezing the GUI; display measurable progress and offer Cancel.	Must

7. Import Preview and Confirmation

Selecting or dropping a valid folder starts discovery only. No copy may begin until the preview is complete and the user confirms Import.

- Show the selected absolute source path.
- Show counts of valid JPEGs, invalid/unreadable JPEGs, hidden/system skips, and filename collisions requiring renaming.
- Show estimated total copy bytes and the destination intake/unsorted path.
- Show Import and Cancel actions.
- Explain that Intake copies files, leaves originals unchanged, and flattens copies into intake/unsorted.
- Block Import when available destination space is less than the calculated valid-source total; show required and available space.

8. Copy Behavior and Filename Collisions

ID	Requirement	Priority
CPY-01	Copy each accepted JPEG byte-for-byte into intake/unsorted while preserving modification time and basic metadata where Windows permits.	Must
CPY-02	Preserve JPEG EXIF data. Apply EXIF orientation only to display; never re-encode imported or sorted files.	Must
CPY-03	Resolve a destination-name collision by appending _1, _2, and higher suffixes before the extension. Never overwrite an existing entry.	Must
CPY-04	Process copying off the UI thread, show file/byte progress at least once per second when measurable, and provide Cancel.	Must
CPY-05	Disable conflicting filesystem-changing Intake controls while scan, copy, recount, recycle, rename, or move operations are active.	Must

9. Import Failures, Cancellation, and Results

APPROVED SIMPLIFIED BEHAVIOR

This release intentionally does not stage or roll back imports. Files copied before a systemic failure or user cancellation remain in intake/unsorted and enter the queue if their manifest records were saved.

- A candidate that becomes unreadable during copy is skipped; the remaining batch continues.

- A systemic destination, storage, access, or persistence failure stops the import and displays an error pop-up.
- A cancelled copy uses the same partial-import behavior as a failed import.
- The warning recommends deleting everything in intake/unsorted before reattempting or sorting. It must explicitly state that this also removes earlier unsorted imports but does not affect intake/sorted.
- The result remains visible until Start Sorting, Import Another Folder, or Back to Intake is selected.

The final summary must show valid discovered, copied, invalid/unreadable skipped, failed during copying, collision-renamed, total bytes copied, and destination path. Skipped, failed, and renamed details must be expandable and use original source-relative paths so duplicate source filenames are distinguishable.

10. Minimal Intake Manifest

A private Intake manifest is required only to operate the queue. It is not the broader cross-app image metadata system.

- For each imported queue item, record its current unsorted filename, batch identifier, batch timestamp, original source-relative path, and queue order.
- Bind manifests to the project's stable internal project ID rather than its mutable name or path.
- Persist mappings and manifests in private per-user application data, not inside user-visible projects.
- Write manifest changes atomically using a temporary file and replacement.
- Update project references when the project is renamed or the complete project library moves.
- Files manually placed in intake/unsorted are not registered and do not appear in the GUI.
- A registered file missing or renamed outside the app is silently omitted from the active queue.

10.1 Manifest Recovery

A corrupt or unreadable manifest blocks sorting and presents Repair Intake Queue. Repair registers direct JPEGs currently in intake/unsorted as one recovered batch. The app must disclose that original source paths and historical batch ordering cannot be recovered.

11. Queue Ordering and Resume

1. Process older import batches before newer batches.
2. Within each batch, order by final collision-safe filename using case-insensitive Windows-style comparison.
3. A skipped image moves behind every other image currently in the complete queue, including newer batches.
4. Remaining manifest-registered files resume automatically the next time Intake opens.
5. Files that no longer exist are silently omitted. Files manually added to unsorted are never appended automatically.

12. Class Discovery and Validation

- Every direct child folder of intake/sorted is discovered as a class.
- Class names are 1-25 characters and must satisfy the existing Windows-safe project-name rules, including invalid characters, reserved device names, trailing space/period, and case-insensitive uniqueness.
- Nested folders are not classes. Review ignores them and shows a warning that only direct JPEG files count.
- More than ten class folders may exist. Only classes with valid unique 0-9 mappings are available for sorting.
- New classes remain unmapped until the user explicitly assigns a free key.

13. Class Management and Key Mapping

ID	Requirement	Priority
CLS-01	Create a new physical class folder only after revalidating its name and confirming no filesystem entry already uses it.	Must
CLS-02	Rename a class by renaming its physical folder. The existing key mapping follows the class after success.	Must
CLS-03	Display ten assignment slots in 0-9 order and a separate list of unmapped classes.	Must
CLS-04	Each key maps to at most one class; each mapped class uses exactly one key. Reject duplicate assignments immediately.	Must
CLS-05	Allow mapping changes only on the landing screen outside an active sorting session; persist them per project.	Must
CLS-06	Do not silently reassign keys when folders are discovered, created, renamed, or deleted.	Must

14. Class Deletion

- An empty class can be deleted with a normal confirmation.
- A nonempty class requires the exact, case-sensitive phrase DELETE <class folder name>.
- The confirmation shows class name, full path, direct JPEG count, total file count including nested/non-JPEG content, and warns that the mapping will be removed.
- Deletion sends the specific class folder and all contents to the Windows Recycle Bin.
- If recycling is unavailable or fails, stop and report the failure. Never fall back to irreversible deletion.
- Remove the mapping only after successful recycling. If deletion fails, folder and mapping remain unchanged.

15. Sorting Screen and Image Viewer

The sorting screen must retain the proven viewer behavior from jpg_sorter.py while adopting the integrated app theme.

- Display the EXIF-corrected image scaled to fit available space without cropping or aspect-ratio distortion.
- Dynamically rerender on window resize with a short debounce; do not change image bytes.
- Show current filename, dimensions, queue position, complete remaining count, session-start total, sorted-this-session count, batch date/time, and original source-relative path.
- Keep mapped class buttons visible, in 0-9 order, labeled with both key and full class name.
- Provide Undo, Skip, Open Image Externally, and <- Back to Project.
- Opening externally uses the Windows default image viewer and does not move or advance the image. Failure is non-blocking.
- A queued file that can no longer render shows an error but may still be classified, skipped, or opened externally.

16. Classification, Skip, and Focus Safety

ID	Requirement	Priority
SRT-01	One press of a mapped 0-9 key or one mapped-class button click immediately moves the current image into that physical class folder.	Must
SRT-02	Never overwrite a class file; use the same collision-safe suffix behavior as import.	Must
SRT-03	An unmapped number key does not move the file and shows brief feedback: No class assigned to key N.	Must
SRT-04	Keyboard classification works only while Intake is foreground, the sorting screen is active, and no text field or dialog has input focus.	Must
SRT-05	Skip changes only queue order, moves no file, and does not enter Undo history.	Must

ID	Requirement	Priority
SRT-06	If every mapped class becomes missing or inaccessible, stop sorting, return to landing, and show a clear error. Do not recreate classes automatically.	Must

17. Multi-Step Undo and Session Boundaries

- Maintain a last-in, first-out history of every successful classification in the active session.
- Allow repeated Undo through all classifications made since Start/Resume Sorting.
- Undo restores the image to intake/unsorted and returns it to the current viewer. If the original unsorted filename is occupied, restore with a safe suffix and explain the rename.
- Skip is never undoable because no filesystem move occurred.
- Back to Project and application close end the session and clear Undo history.
- Starting another import or opening Review requires ending the active sorting session first.
- Leaving with Undo history requires confirmation: Leaving Intake will clear the Undo history. Sorted files will remain in their class folders.
- At queue completion, retain Undo until the user chooses a completion next step.

18. Completion and Review Master Dataset

18.1 Completion

When the queue reaches zero, show counts moved into each class during the session and these next-step actions: Import More Images, Review Master Dataset, and Return to Project Tools.

18.2 Review Master Dataset

- Use a single summary page with the total direct JPEG count for each direct class folder and a total master-dataset count.
- List classes in 0-9 mapped order followed by unmapped classes.
- Include a clickable Windows File Explorer link for intake/sorted and for every class folder.
- Count manually added direct .jpg/.jpeg files because Review describes the physical master dataset.
- Ignore nested folders and their contents; show a non-blocking nested-folder warning.
- Provide Refresh and show a loading state while recounting.
- Do not provide thumbnails, image opening, filtering, reclassification, or editing in this release.

19. Project Statistics and Last Edited

- Preserve the existing project Total images definition: recursively count valid .jpg/.jpeg files across every folder within the project.
- Refresh total images and last edited after imports (including partial/cancelled copies), classification, Undo, class recycle, and returning to the project screen.
- Run recounting without freezing the GUI.
- Do not change labeled-image count or labeling progress. Intake classification is not bounding-box labeling.
- Continue to support the existing manual home-page Refresh behavior.

20. Loading, Error, and Accessibility Requirements

- Keep the app responsive during scan, validation, copy, image decoding, recount, class recycling, and other measurable work.
- Present specific errors with the failed operation and relevant path; never report success before the filesystem and private-state operations succeed.
- Use the existing minimum window size. At small sizes, scroll configuration/summary regions while preserving a usable primary viewer.
- Provide visible focus, logical Tab order, keyboard activation, readable contrast, text labels in addition to color, dialog focus containment, Escape for non-destructive dialogs, and explicit destructive confirmation.
- Do not connect to network services, collect telemetry, or use cloud storage.

21. State Transitions

From	Event	To	Required behavior
Project Tools	Click Intake card	Intake Landing	Create/reuse required folders and load classes, mappings, manifest, and queue.
Intake Landing	Drop/browse folder	Scanning	Validate exactly one folder; scan recursively off the UI thread.
Scanning	Complete	Import Preview	No files copied yet.
Import Preview	Confirm Import	Copying	Block if free space is insufficient.
Copying	Success/partial/cancel	Import Results	Persist successful queue records; disclose partial state.
Intake Landing/Results	Start/Resume	Sorting	Requires queue and at least one valid mapping.
Sorting	Classify	Sorting/Complete	Move one file, add Undo record, advance queue.
Sorting	Skip	Sorting	Move item behind complete queue; no filesystem move.
Sorting	Back	Project Tools	Confirm if Undo history exists; clear session history.
Complete	Review	Dataset Review	Retain Undo until the completion action is chosen.

22. Suggested Architecture Boundaries

- Project workspace/navigation: project-tools view, tool cards, Intake entry/exit, and shared theme.
- Intake filesystem service: folder creation, discovery, validation, copy, collision naming, move, Undo, class rename, and safe recycle.
- Manifest store: atomic per-project manifest and mapping persistence keyed by stable project ID.
- Queue service: deterministic batch ordering, Skip reordering, resume, and missing-file omission.
- Background work coordinator: scan, copy, validation, statistics recount, cancellation, progress, and UI marshaling.
- Viewer: Pillow-based EXIF transpose and aspect-ratio-preserving responsive rendering, adapted from `jpg_sorter.py`.
- Statistics integration: reuse existing project scan/update contracts; Intake must not redefine labeled progress.

Filesystem and manifest logic should be testable without launching Tkinter. Destructive targets must be resolved and revalidated immediately before acting. Background workers must not mutate Tk widgets directly.

23. Acceptance Criteria

1. Opening a project shows a clickable Intake card with title and short description, without status or a separate Open button.
2. First opening Intake creates intake/unsorted and intake/sorted without overwriting existing contents.
3. A user can drop or browse exactly one local Windows folder and recursively scan nested JPEGs without freezing the UI.

4. Hidden/system entries and reparse-point directories are skipped; uppercase JPEG extensions are accepted when content is valid.
5. Preview shows path, valid/invalid/skipped/collision counts, bytes, destination, and a copy-not-move confirmation.
6. Import is blocked with a clear required/available-space message when destination capacity is insufficient.
7. Confirmed valid images are copied byte-for-byte into flattened unsorted storage; originals remain unchanged.
8. Filename collisions produce _1, _2, and higher suffixes and never overwrite files.
9. Invalid or newly unreadable files are skipped and identified by source-relative path while other valid files continue.
10. Systemic failure or cancellation retains successful partial copies, registers them, and warns about clearing all unsorted content.
11. The result summary contains every approved count, byte total, destination, and expandable detail list.
12. Private atomic manifests preserve older-batch-first and per-batch alphabetical order across restarts.
13. Manually added unsorted files do not appear; missing registered files are silently omitted.
14. Manifest corruption blocks sorting and Repair Intake Queue creates one disclosed recovered batch.
15. Direct sorted child folders appear as classes; valid class names use the approved 1-25-character Windows rules.
16. Ten mapping slots appear in 0-9 order, keys are exclusive, and unmapped classes remain valid but unavailable for sorting.
17. Class creation, rename, delete, and mapping operations change real folders/state only after successful validation.
18. Nonempty class recycling requires exact DELETE <class name>; failure never falls back to permanent deletion.
19. The sorting viewer preserves EXIF orientation and aspect ratio and dynamically fits when the window is resized.
20. One mapped keypress/button click moves one image safely; unmapped keys produce brief feedback and no move.
21. Skip places the image after the complete current queue and creates no Undo entry.
22. Multi-step LIFO Undo restores every move from the active session and resolves restoration collisions safely.
23. Focus safeguards prevent classification while another window, text input, or dialog has focus.
24. Leaving with Undo history warns the user and clears history only after confirmed exit.
25. Completion shows per-class session counts and the three approved next-step actions.
26. Review shows direct JPEG counts, total master count, sorted/class Explorer links, refresh, and nested-folder warnings.
27. Project Total images remains a responsive recursive count across the project; labeled progress is unchanged.
28. The app remains local and offline; README states that only local Windows folders are supported.
29. The legacy jpg_sorter.py and run_jpg_sorter.bat remain functional and unchanged.

24. Required Tests

24.1 Automated

- Recursive case-insensitive JPEG discovery; invalid content; hidden/system exclusions; reparse-point exclusion and cycle safety.
- Flattened copy, metadata preservation where supported, source immutability, and deterministic collision suffixing.
- Free-space blocking, individual-file skip, systemic partial failure, cancellation, and results aggregation.
- Atomic manifest save/load, stable project-ID binding, move/rename updates, corruption detection, and queue repair.
- Older-batch-first ordering, case-insensitive final-name ordering, full-queue Skip, resume, missing omission, and manual-file exclusion.
- Class-name validation, discovery, mapping exclusivity, persistence, folder rename, empty recycle, and typed nonempty recycle validation.

- Classification moves, destination collision handling, multi-step LIFO Undo, Undo collision handling, and session-history clearing.
- Recursive project statistics remain correct after import, move, Undo, recycle, and external filesystem changes.
- Source images are never modified or deleted during preview or import.

24.2 Manual GUI Verification

- Project tool-card integration, landing layout, real Windows drag-and-drop, picker fallback, scan/copy progress, cancellation, and disabled states.
- Responsive viewer at minimum and larger window sizes; no clipping, overlap, aspect-ratio change, or stale image after rapid resize.
- EXIF orientation, corrupt-image viewer state, external-viewer success/failure, and no queue advance from external opening.
- Keyboard and button classification, unmapped-key feedback, foreground/focus safeguards, Tab order, visible focus, and dialog behavior.
- Undo warning on exit, completion choices, Review counts/links/refresh, nested-folder warning, and File Explorer opening.
- Windows Recycle Bin success and failure paths without irreversible fallback.
- README local-folder limitation and unchanged legacy sorter launch/use.

25. Risks, Assumptions, and Future Enhancements

Type	Item	Resolution / Boundary
Risk	Partial imports can mix with earlier unsorted batches because staging/rollback is deferred.	Mitigate with explicit warning and detailed results; transactional staging is future work.
Risk	Manual filesystem edits can make the minimal manifest diverge from disk.	Omit missing items, exclude unregistered additions, and provide manifest repair.
Risk	Recycle Bin behavior varies by filesystem and Windows configuration.	Treat unsupported recycle as failure; never permanently delete as fallback.
Assumption	Projects retain stable internal IDs across rename and library relocation.	Bind Intake private state to that existing identity.
Assumption	Pillow remains available for JPEG verification and display.	Retain friendly dependency handling and add requirements/tests as needed.
Future	Transactional staging with verified all-or-nothing finalization and automatic rollback.	Explicitly excluded from this release.
Future	Cross-app image lineage, content fingerprints, duplicate detection, and additional formats.	Scope separately after Intake integration.
Future	Advanced master-dataset thumbnails, navigation, filtering, and reclassification.	Review remains summary-only in this release.
Future	Deprecation of the standalone legacy JPEG sorter.	Leave current files and launcher operational for now.

IMPLEMENTATION GATE

This PRD is Pending Approval. Implementation should begin only after approval or explicit authorization from the product owner.